

UNIT VI

Contents

Semantic Analysis

Syntax directed translation

S- Attributed

L Attributed definition

Type checker

Intermediate code generation

3-address code form

DAG

Code optimization

Optimization

loop optimization

peep-hole optimization

Symbol table format

1. Grammar symbols are associated with **attributes** to associate information with the programming language constructs that they represent.
2. Values of these attributes are evaluated by the **semantic rules** associated with the production rules.
3. An attribute may hold almost any thing.
 - A value, a type, a string, a number, a memory location, a complex record.

Syntax-Directed Definitions and Translation Schemes

1. When we associate semantic rules with productions, we use two notations:

- **Syntax-Directed Definitions**
- **Translation Schemes**

A. Syntax-Directed Definitions:

- give high-level specifications for translations
- hide many implementation details such as order of evaluation of semantic actions.
- We associate a production rule with a set of semantic rules, and we do not say when they will be evaluated.

B. Translation Schemes:

- indicate the order of evaluation of semantic actions associated with a production rule.
- In other words, translation schemes give a little bit information about implementation details.

Syntax-Directed Definitions

1. A syntax-directed definition is a generalization of a context-free grammar in which:
 - Each grammar symbol is associated with a set of attributes.
 - This set of attributes for a grammar symbol is partitioned into two subsets called
 - **synthesized** and
 - **inherited** attributes of that grammar symbol.
 - Each production rule is associated with a set of semantic rules.

Syntax-Directed Definitions

1. The value of a **synthesized attribute** at a node is computed from the values of attributes at the *children* of that node in the parse tree.
2. The value of an **inherited attribute** is computed from the values of attributes at the *siblings* and/or a *parent* of that node.
3. Evaluation of a semantic rule defines the value of an attribute.

Syntax-Directed Definition

In a syntax-directed definition, each production $A \rightarrow \alpha$ is associated with a set of semantic rules of the form:

$$b = f(c_1, c_2, \dots, c_n)$$

where f is a function and b can be one of the followings:

→ b is a **synthesized attribute** of A and $c_1, c_2, \dots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

OR

→ b is an **inherited attribute** of one of the grammar symbols in α (on the right side of the production), and $c_1, c_2, \dots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

Syntax-Directed Definition – Synthesized attributes

Production

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$E.val = E_1.val + T.val$

$E.val = T.val$

$T.val = T_1.val * F.val$

$T.val = F.val$

$F.val = E.val$

$F.val = \text{digit.lexval}$

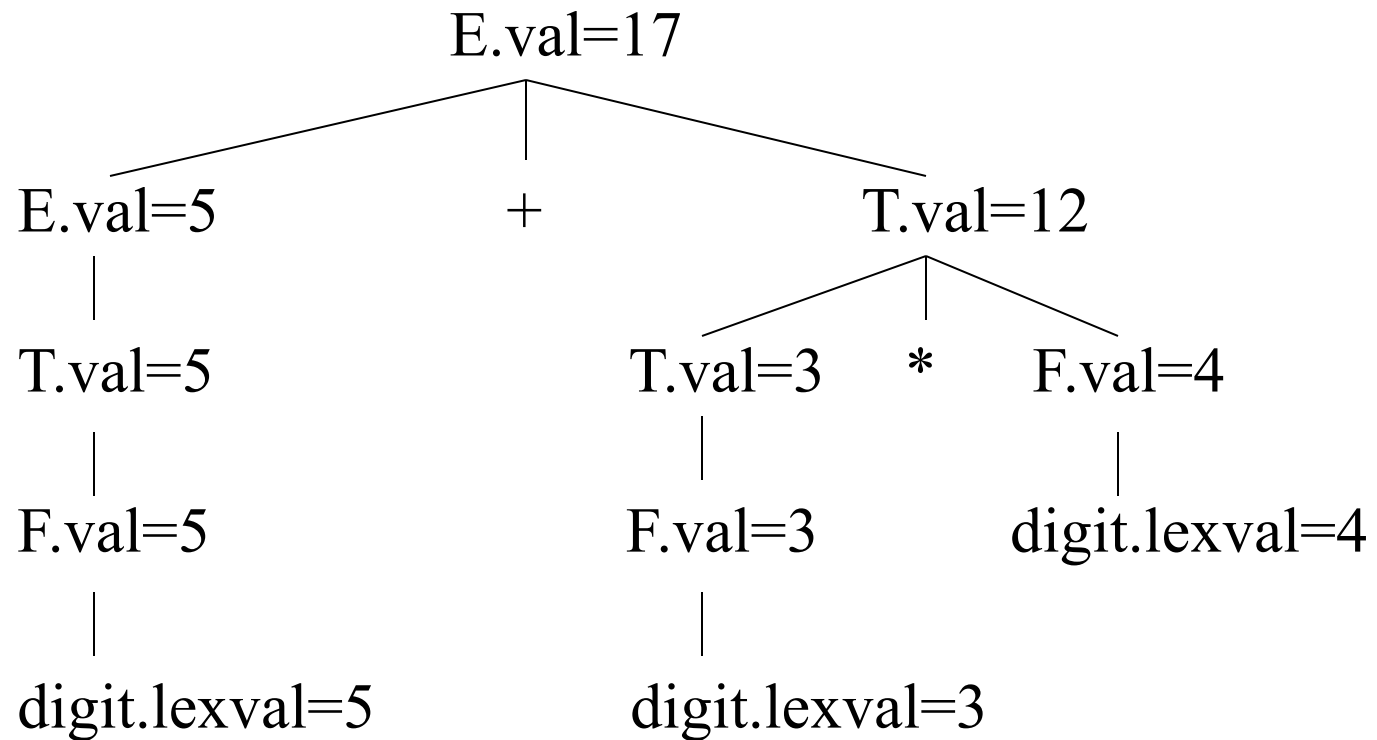
1. Symbols E, T, and F are associated with a synthesized attribute *val*.
2. The token digit has a synthesized attribute *lexval* (it is assumed that it is evaluated by the lexical analyzer).

Annotated Parse Tree

1. A parse tree showing the values of attributes at each node is called an **annotated parse tree**.
2. The process of computing the attributes values at the nodes is called **annotating** (or **decorating**) of the parse tree.

Annotated Parse Tree -- Example

Input: $5+3*4$



Dependency graph

The interdependencies among the inherited and synthesized attributes at the nodes in a parse tree can be depicted by a directed graph called dependency graph

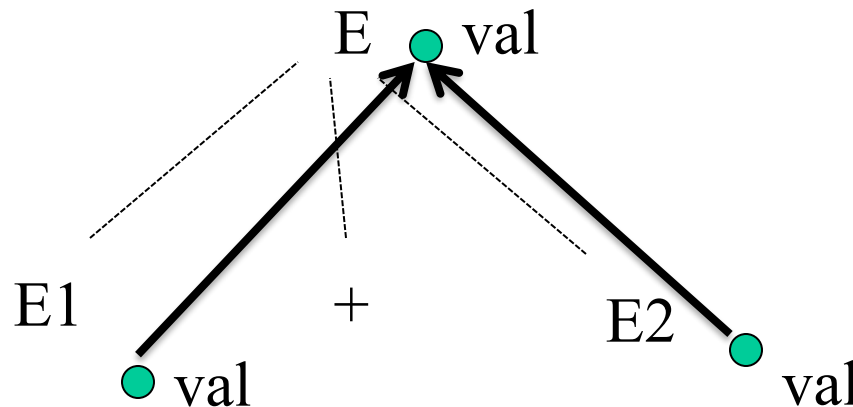
dependency graph for the below production is:

Production

$E \rightarrow E1 + E2$

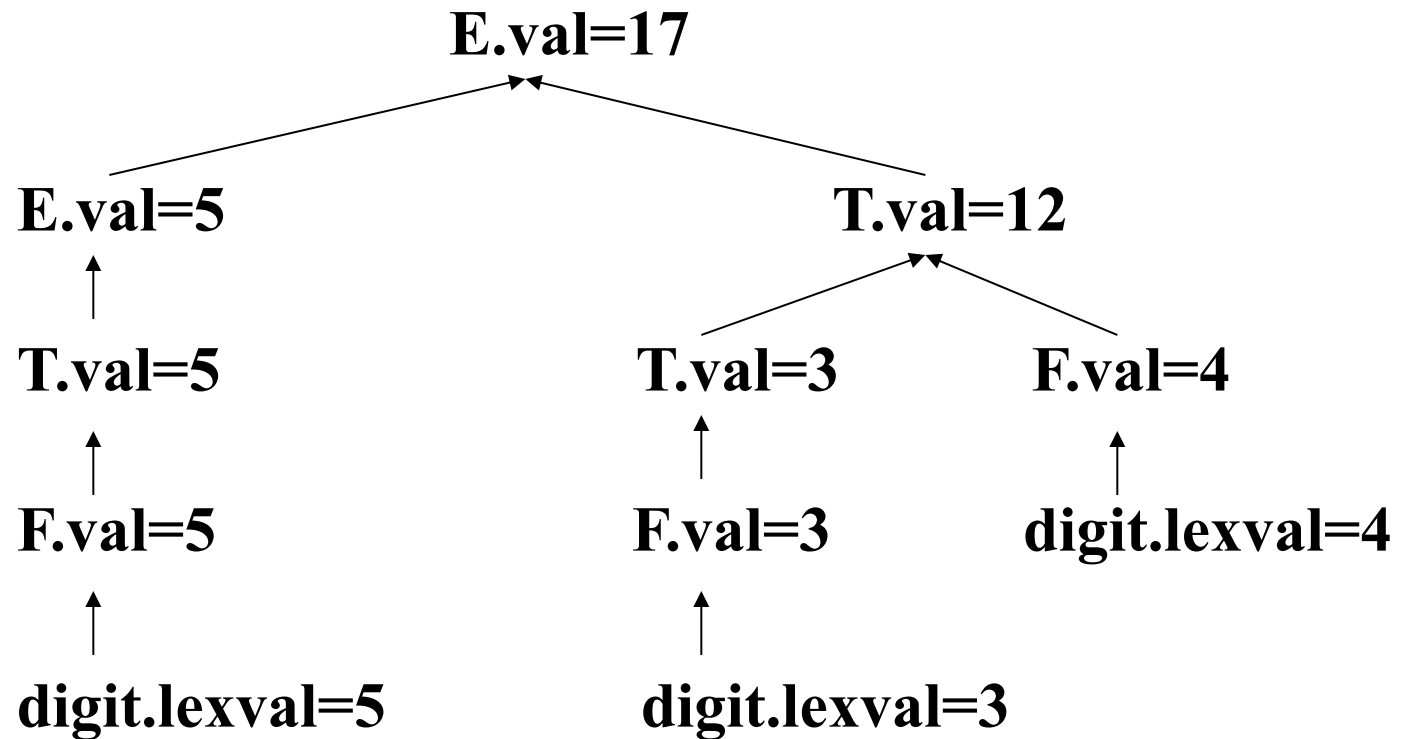
semantic rule

$E.val = E1.val + E2.val$



Dependency Graph

Input: $5+3*4$



Syntax-Directed Definition – Inherited Attributes

Production

Semantic Rules

$D \rightarrow T L$

$L.in = T.type$

$T \rightarrow \mathbf{int}$

$T.type = \text{integer}$

$T \rightarrow \mathbf{real}$

$T.type = \text{real}$

$L \rightarrow L_1, \mathbf{id}$

$L_1.in = L.in$

$\text{addtype}(\mathbf{id.entry}, L.in)$

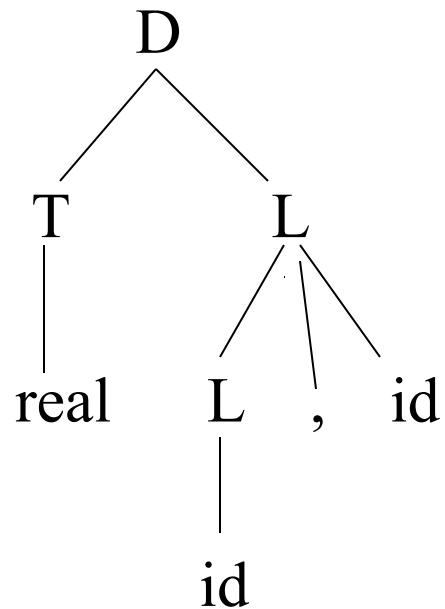
$L \rightarrow \mathbf{id}$

$\text{addtype}(\mathbf{id.entry}, L.in)$

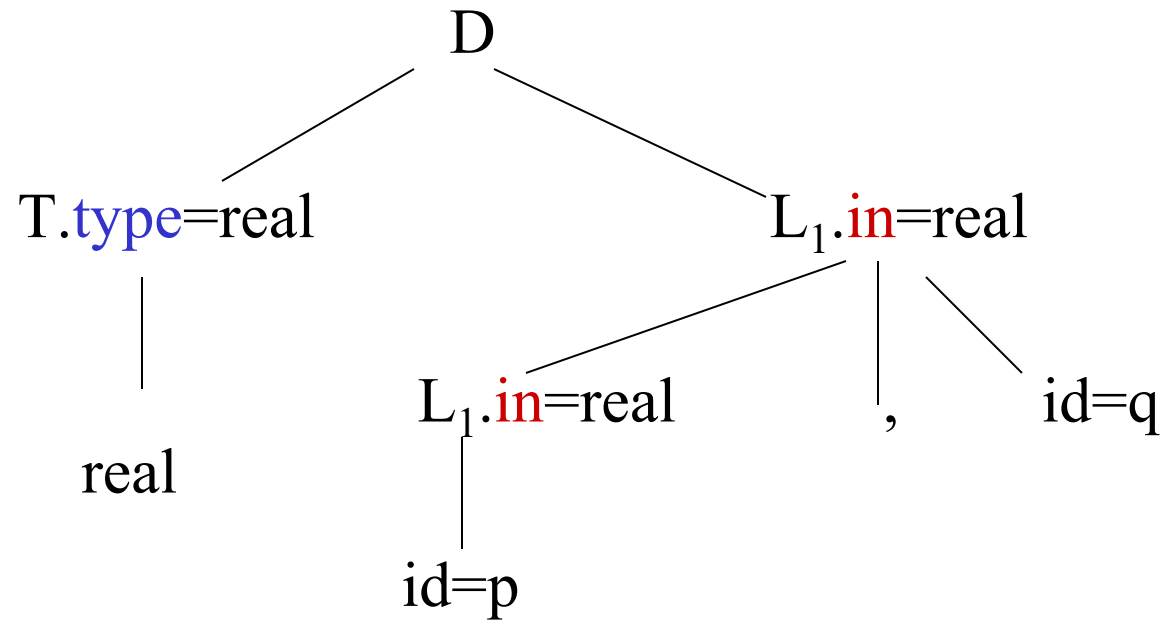
1. Symbol T is associated with a synthesized attribute *type*.
2. Symbol L is associated with an inherited attribute *in*.

Inherited Attributes

Input: `real p , q`



parse tree



Annotated parse tree

S-Attributed Definitions

1. Syntax-directed definitions are used to specify syntax-directed translations.
2. We would like to evaluate the semantic rules during parsing (i.e. in a single pass, we will parse and we will also evaluate semantic rules during the parsing).
3. We will look at two sub-classes of the syntax-directed definitions:
 - **S-Attributed Definitions:** only synthesized attributes used in the syntax-directed definitions.
 - **L-Attributed Definitions:** in addition to synthesized attributes, we may also use inherited attributes in a restricted fashion.
4. Implementations of S-attributed Definitions are a little bit easier than implementations of L-Attributed Definitions

S-attributed Definition

Production

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$E.val = E_1.val + T.val$

$E.val = T.val$

$T.val = T_1.val * F.val$

$T.val = F.val$

$F.val = E.val$

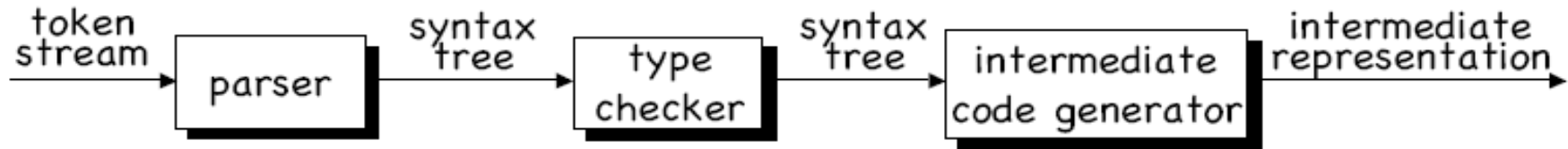
$F.val = \text{digit.lexval}$

1. Symbols E, T, and F are associated with a synthesized attribute *val*.
2. The token digit has a synthesized attribute *lexval* (it is assumed that it is evaluated by the lexical analyzer).

L-Attributed Definitions

- A syntax-directed definition is called *L-Attributed* if
 - In the semantic rules of a production $A \rightarrow X_1 \dots X_n$ the inherited attributes of X_j depend only on
 - The attributes (synthesized or inherited) of the symbols $X_1 \dots X_{j-1}$
 - The inherited attributes of A
- All S-attributed Definitions are also L-attributed because the restrictions apply only to inherited attributes.

Type Checking



- TYPE CHECKING is the main activity in semantic analysis.
- Goal: calculate and ensure consistency of the type of every expression in a program
- If there are type errors, we need to notify the user.
- Otherwise, we need the type information to generate code that is correct.

Type Expression

- The type of a language construct is denoted by a *type expression*.
- In general a *type expression* can be either a basic type or derived type ,in the case of derived types (like arrays ,pointers etc) , type constructor is applied:
 - **A basic type**
 - a primitive data type such as *integer, real, char, boolean, ...*
 - **A type constructor applies to other type expressions.**
 - **arrays:** If T is a type expression, then $array(I,T)$ is a type expression where I denotes index range.
Ex: $array(0..99,int)$
 - **products:** If T_1 and T_2 are type expressions, then their cartesian product $T_1 \times T_2$ is a type expression.
Ex: $int \times int$
 - **pointers:** If T is a type expression, then $pointer(T)$ is a type expression.
Ex: $pointer(int)$

Type Expression

- **Functions:** We may treat functions in a programming language as mapping from a domain type D to a range type R .
- So, the type of a function can be denoted by the type expression $D \rightarrow R$ where D and R are type expressions.
- Ex: $\text{int} \rightarrow \text{int}$ represents the type of a function which takes an int value as parameter, and its return type is also int.
 - **Function** `int sum(int a,int b)`
 - **Type expression** for sum is $\text{int} \times \text{int} \rightarrow \text{int}$

Type Expression

Struct: The structure given by keyword struct is also a type expression. It is applied as a product of type of the fields(members) of the structure.

Ex: struct stud{
 char name[10];
 float marks;
}

stud student[10];

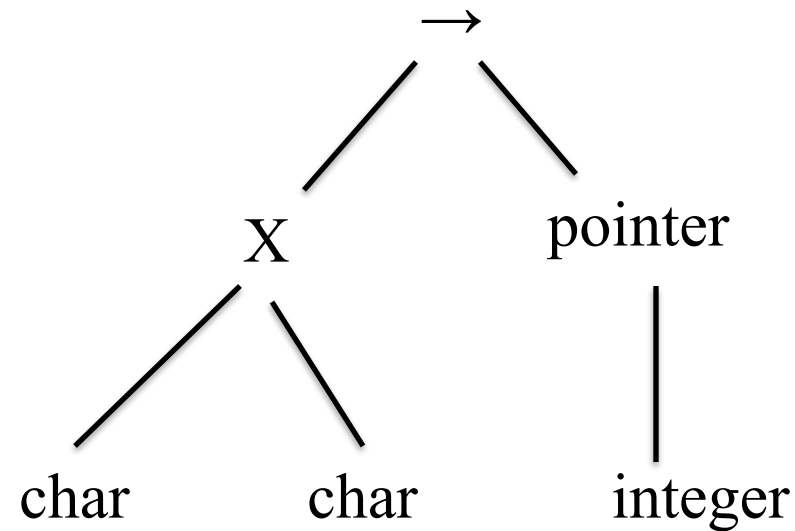
Here,type name is stud having type expression as

Struct((name x array(0,1,...9,char))x(marks float))

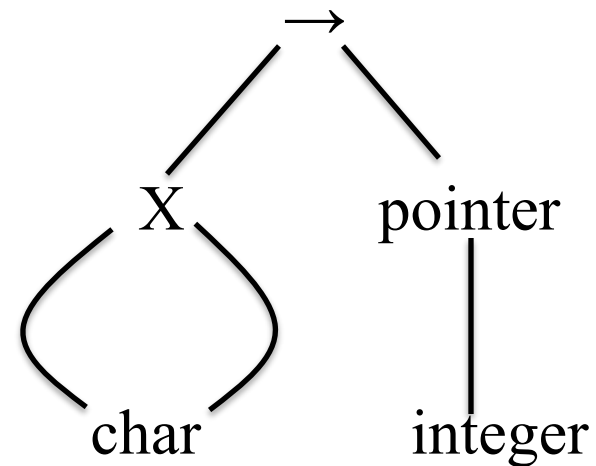
Thus, type expression for student is array(0,1,...9,stud)

Graphical representation of type expressions

- Type expressions are represented using syntax tree or dag.
- Consider a type expression is : **char X char $\rightarrow$ pointer (integer)**



Synax tree



DAG

Specification of simple type checker

Grammar for simple language

$P \rightarrow D; E$

$D \rightarrow T \text{ List}$

$T \rightarrow \text{char} / \text{int} / \text{float}$

$\text{List} \rightarrow L_1[\text{num}] /* L_1 / \text{id}$

$E \rightarrow \text{literal}$

$E \rightarrow \text{num}$

$E \rightarrow \text{id}$

$E \rightarrow E \text{ mod } E$

$E \rightarrow E[E]$

$E \rightarrow E \text{ op } E$

$E \rightarrow *E$

Ex: char *x;

int a[10];

float a;

A Simple Type Checking System

semantic actions

$P \rightarrow D;E$

$D \rightarrow T \text{ List}$

$\text{List} \rightarrow \text{id}$

$T \rightarrow \text{char}$

$T \rightarrow \text{int}$

$T \rightarrow \text{float}$

$\text{List} \rightarrow * L_1$

$\text{List} \rightarrow L_1[\text{num}]$

$\{\text{List.type} = T.\text{type}\}$

$\{\text{addtype}(\text{id.entry}, \text{List.type})\}$

$\{\text{T.type} = \text{char}\}$

$\{\text{T.type} = \text{int}\}$

$\{\text{T.type} = \text{float}\}$

$\{L_1.\text{type} = \text{pointer}(\text{List.type})\}$

$\{L_1.\text{type} = \text{array}(0..\text{num.val}-1, \text{List.type})\}$

**Parts of translation scheme that saves the
type of an identifier in symbol table**

Type Checking of Expressions

$E \rightarrow \text{num} \quad \{ E.\text{type}=\text{integer} \}$

$E \rightarrow \text{literal} \quad \{ E.\text{type}=\text{char} \}$

$E \rightarrow \text{id} \quad \{ E.\text{type}=\text{lookup}(\text{id}.\text{entry}) \}$

**$E \rightarrow E_1 + E_2 \quad \{ \text{if } (E_1.\text{type}=\text{int} \text{ and } E_2.\text{type}=\text{int}) \text{ then } E.\text{type}=\text{int}$
 else if (E₁.type=int and E₂.type=real) then E.type=real
 else if (E₁.type=real and E₂.type=int) then E.type=real
 else if (E₁.type=real and E₂.type=real) then E.type=real
 else E.type=type-error } }**

Type Checking of Expressions

$E \rightarrow E_1 \text{ mod } E_2$ { if ($E_1.\text{type}=\text{int}$ and $E_2.\text{type}=\text{int}$)
 then $E.\text{type}=\text{int}$
 else $E.\text{type}=\text{type-error}$ }

$E \rightarrow E_1 [E_2]$ { if ($E_2.\text{type}=\text{int}$ and $E_1.\text{type}=\text{array}(s,t)$)
 then $E.\text{type}=t$
 else $E.\text{type}=\text{type-error}$ }

$E \rightarrow E_1 \uparrow$ { if ($E_1.\text{type}=\text{pointer}(t)$)
 then $E.\text{type}=t$
 else $E.\text{type}=\text{type-error}$ }

Type Checking of Statements

$S \rightarrow id = E \quad \{ \text{if } (id.type=E.type \text{ then } S.type=void$
 $\text{else } S.type=type-error \}$

$S \rightarrow \text{if } E \text{ then } S_1 \quad \{ \text{if } (E.type=boolean \text{ then } S.type=S_1.type$
 $\text{else } S.type=type-error \}$

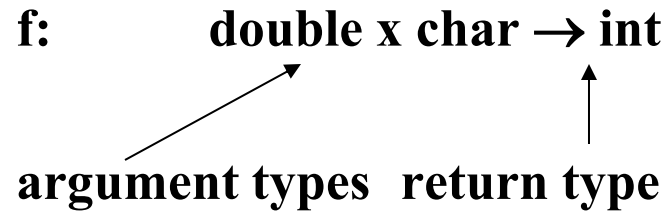
$S \rightarrow \text{while } E \text{ do } S_1 \quad \{ \text{if } (E.type=boolean \text{ then } S.type=S_1.type$
 $\text{else } S.type=type-error \}$

Type Checking of Functions

$E \rightarrow E_1 (E_2) \quad \{ \text{if } (E_2.\text{type}=s \text{ and } E_1.\text{type}=s \rightarrow t) \text{ then } E.\text{type}=t$
 $\quad \text{else } E.\text{type}=\text{type-error} \}$

Ex: `int f(double x, char y) { ... }`

f: `double x char → int`



argument types return type

Equivalence of Type Expressions

- **Checking Rules:**

if two type expressions are equal then return a certain type else return a `type_error`

- It is important to have a precise definition of when two type expressions are equivalent.

The notion of type equivalence implemented by a specific compiler can be explained using the concepts of

- Structural equivalence
- Name equivalence

Structural Equivalence of Type Expressions

1. Two type expressions are structural equivalent if they are
 - same basic type or
 - formed by applying the same constructor
2. That is, two type expressions are structural equivalent if and only if they are identical.
3. Example:
 - *integer* is equivalent to *integer*
 - *pointer(char)* is equivalent to *pointer(char)*

Structural Equivalence of Type Expressions

```
function sequiv(s, t): boolean;  
begin  
    if s and t are the same basic type return true  
    else if s = array (s1, s2) and t = array(t1, t2) then  
        return sequiv(s1, t1) and sequiv(s2, t2)  
    else if s = s1 × s2 and t = t1 × t2 then  
        return sequiv(s1, t1) and sequiv(s2, t2)  
    else if s = pointer(s1) and t = pointer(t1) then  
        return sequiv(s1, t1)  
    else if s = s1 → s2 and t = t1 → t2 then  
        return sequiv(s1, t1) and sequiv(s2, t2)  
    else return false  
end
```

Encoding of Type Expression

- The encoding of type expression is from C (D. M. Ritchie)
- Consider the type constructors
 - **Array**
 - **Pointer**
 - **Function**
- Say
 - **pointer(t):** a pointer to type t
 - **freturns(t):** a function of some arguments that returns an object of type t
 - **array(t):** an array of elements of type t
- Example of such type expressions:
 1. Char
 2. Freturns(char)
 3. Pointer(Freturns(char))
 4. Array(Pointer(Freturns(char)))

Encoding of Type Expression (cont.)

Type Constructor	Encoding
pointer	01
array	10
freturns	11

Type Constructor	Encoding
boolean	0000
char	0001
integer	0010
real	0011

Type Constructor	Encoding
char	000000 0001
freturns(char)	000011 0001
pointer(freturns(char))	000111 0001
array(pointer(freturns(char)))	100111 0001

Encoding of Type Expression (cont.)

- **Advantages**

- **saves space**
- **Keeps track of the constructors that appear in any type expression**
- **Two bit sequences cannot represent the same type because either**
 - **The basic types are different or**
 - **The constructors in the type expressions are different**

1. Disadvantages

- **Different types could have the same bit sequence since array size and function arguments are not represented**

Name for Type Expression

1. Types can be given names.
2. When names are allowed in type expressions, two notions of equivalence of type expressions arise, depending on the treatment of names
3. **Name equivalence** views each type name as a **distinct** type, so two type expressions are name equivalent if and only if they are identical.
4. Under **structural equivalence**, names are replaced by the type expressions they define, so two type expressions are structurally equivalent if they represent two structurally equivalent the expressions when all names have been substituted out.

Name for Type Expression (cont.)

```
Typedef struct Node{  
  int x;  
}node;  
node *first,*second;  
Struct Node  
  *last1,*last2;
```

Variable	Type expression
first	node
second	node
last1	pointer(Node)
last2	Pointer(Node)

Do the variables first, second, last1, last2 all have identical types?

Name equivalence:

- first and second have the same type
- Last1, last2 also have the same type
- But first and last1 do not.

Structural equivalence:

- all four variables have the same type because node is a name for the type expression *pointer(Node)*.

Type Conversions

- Expression $x+i$ where x is real and i is integer.
- The compiler have to convert one of the operands of $+$ to ensure that both operands are of the same type when the addition takes place .

Coercions:

Conversion from one type to another is said to be *implicit* if it is to be done automatically by the computer.

Implicit type conversions are called coercions.

Conversion is said to be explicit if the programmer must write something to cause the conversion.

Type checking rules for Coercions from integer to real

$$E \rightarrow \text{num} \quad \{ E.\text{type} = \text{integer} \}$$
$$E \rightarrow \text{num.num} \quad \{ E.\text{type} = \text{real} \}$$

E $\rightarrow$ **id** { **E.type**=lookup(**id.entry**) }

$$E \rightarrow E_1 \text{ op } E_2 \quad \{ \begin{array}{l} E.\text{type} = \text{if } E_1.\text{type}=\text{integer and } E_2.\text{type}=\text{integer} \\ \quad \text{then integer} \\ \quad \text{else if } E_1.\text{type}=\text{integer and } E_2.\text{type}=\text{real} \\ \quad \quad \text{then real} \\ \quad \text{else if } E_1.\text{type}=\text{real and } E_2.\text{type}=\text{integer} \\ \quad \quad \text{then real} \\ \quad \text{else if } E_1.\text{type}=\text{real and } E_2.\text{type}=\text{real} \\ \quad \quad \text{then real} \\ \quad \text{else type-error} \end{array} \}$$

Overloading of Functions and Operators

- **An overloaded symbol is one that has different meaning depending on its context.**
- **An operator is overloaded if the same operator is used for two different operations.**
- **2+3 represents integer additions**
- **2.1+3.0 represents floating-point addition**
- **Overloading can be extended to user-defined functions.**

int max (int x, int y)

double max (double x, double y)

The type checker can decide which function is meant based on the types of parameters (C++, Ada).

Polymorphic functions

- **An ordinary procedure allows the statements in its body to be executed with arguments of fixed types.**
- **Each time a polymorphic procedure is called , the statements in its body can be executed with arguments of different types.**

Intermediate code Generation

Intermediate languages

Implementation of 3-address statements

Intermediate Code Generation

- ❖ The given program in a source language is converted to an equivalent program in an intermediate language by the intermediate code generator.
- ❖ Intermediate language can be many different languages, and the designer of the compiler decides this intermediate language.

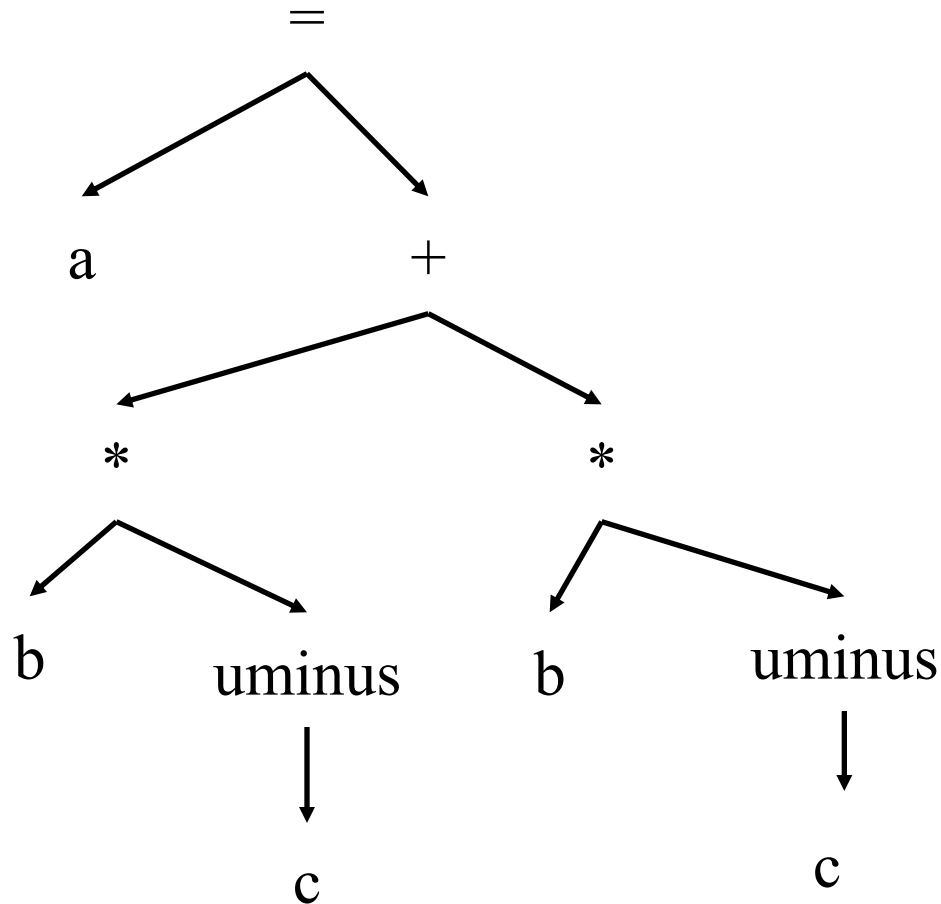
- ❑ syntax trees

- ❑ postfix notation

- ❑ three-address code

Syntax Trees

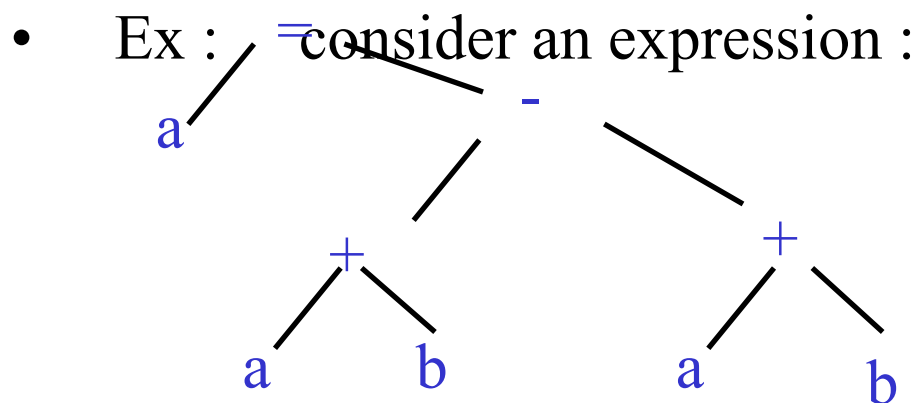
- Consider the assignment **a=b*-c+b*-c**



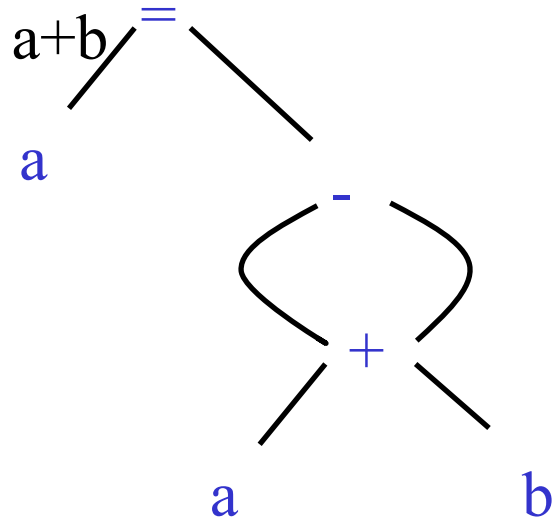
DAG (Directed Acyclic Graph)

- A **dag** is similar to syntax tree , and it identifies the common sub expressions in the expression.
- A node in a dag representing a common sub expression has more than one parent .
- In a syntax tree, the common sub expressions would be represented as duplicated subtree.

• Syntax tree

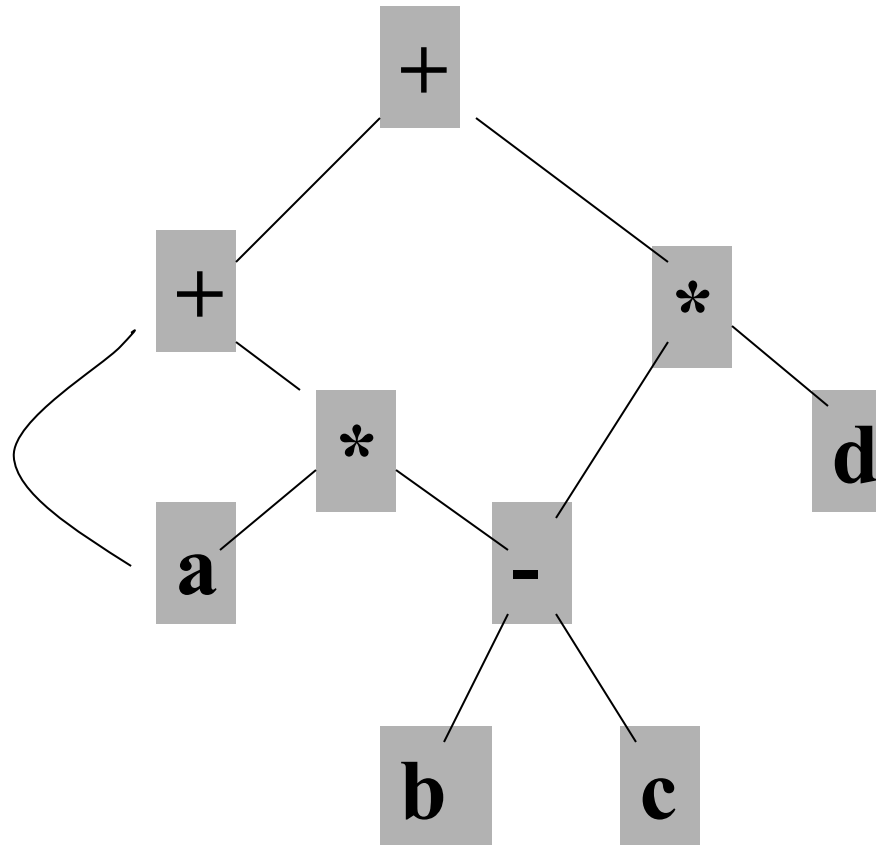


DAG



Directed Acyclic Graphs for Expressions

$a + a * (b - c) + (b - c) * d$



Postfix notation

Postfix notation is a linear representation of a syntax trees.

Consider an expression : $a = b * -c + b * -c$

postfix notation : **$a \ b \ c \ - \ * \ b \ c \ - \ * \ + \ =$**

Constructing Syntax Trees for Expressions

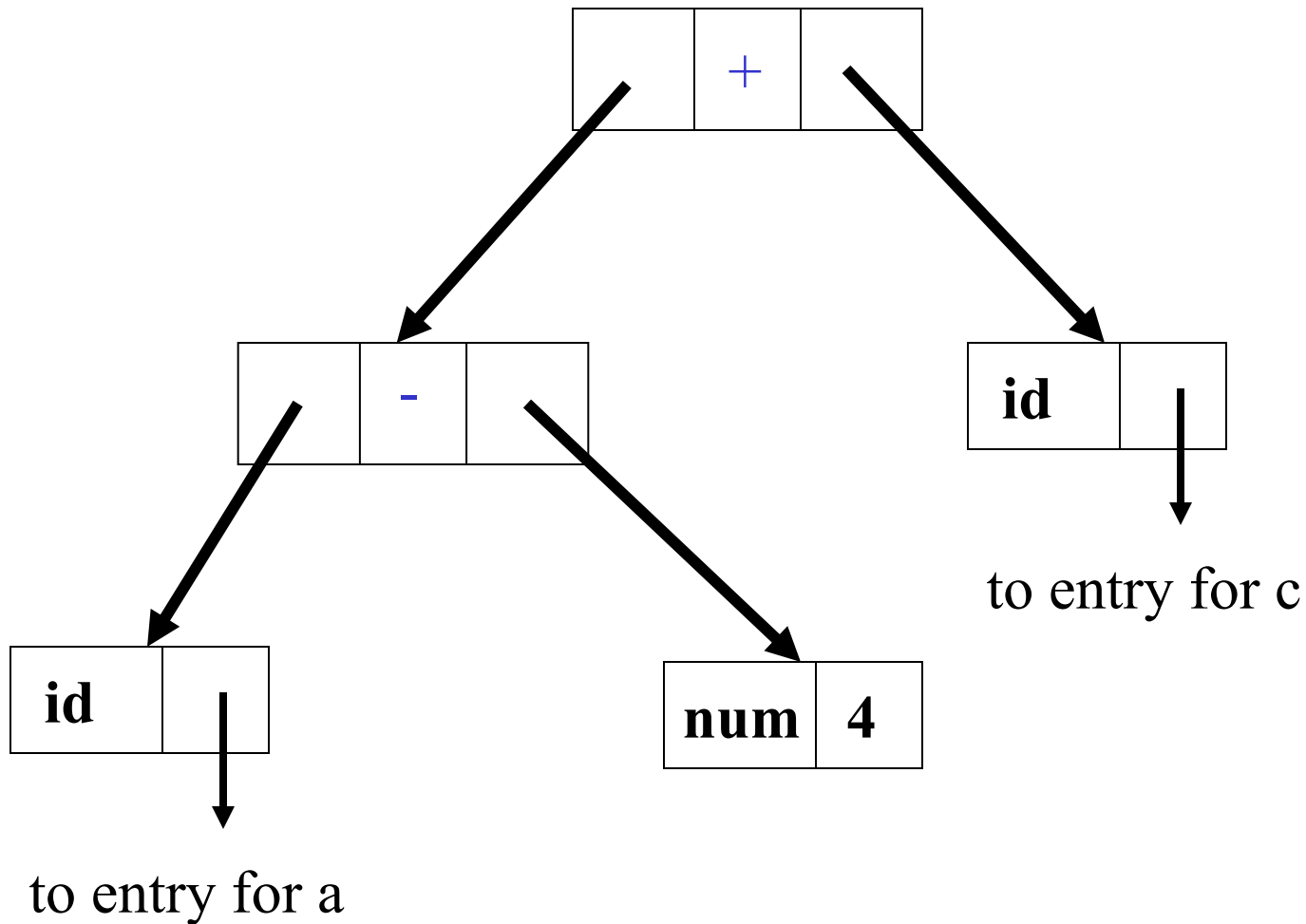
1. Syntax trees are constructed using following functions:
 - i. ***mknode**(**op** , **left**, **right**)* : creates an operator node with label **op** and two fields containing pointers to **left child** and **right child**.
 - ii. ***mkleaf** (**id** ,**entry**)* : creates an identifier node with label **id** and a field containing **entry**, a pointer to the symbol table.
 - iii. ***mkleaf** (**num** , **val**)* : creates a number node with label **num** and a field containing **val** , the value of the number
2. Each function returns a pointer to a newly crated node

The following sequence of function calls creates the syntax tree
for the expression : **a-4+c**

- 1) p1= mkleaf (**id** , entrya);
- 2) p2=mkleaf(**num** , 4);
- 3) p3=mknode('-', p1 ,p2);
- 4) p4=mkleaf(**id** , entryc);
- 5) p5=mknode('+', p3 , p4);
- In this sequence , **p1 ,p2 ,p5** are pointers to nodes
- **entrya** and **entryc** are pointers to symbol table entries for identifiers **a** and **c**
- The syntax tree is constructed bottom up.

Draw the Syntax Tree

a-4+c



SDD for Syntax Trees

PRODUCTION

$E \rightarrow E_1 + T$

$E \rightarrow E_1 - T$

$E \rightarrow T$

$T \rightarrow (E)$

$T \rightarrow \text{id}$

$T \rightarrow \text{num}$

SEMANTIC RULE

$E.nptr = mknnode('+', E_1.nptr, T.nptr)$

$E.nptr = mknnode('-', E_1.nptr, T.nptr)$

$E.nptr = T.nptr$

$T.nptr = E.nptr$

$T.nptr = mkleaf(id, id.entry)$

$T.nptr = mkleaf(num, num.val)$

The synthesized attribute **nptr** for E and T keeps track of the pointers returned by the function calls

Three Address Code

- Three address code is a linear representation of syntax tree
- Statements of general form **$x = y \text{ op } z$**
- Ex: $x = y + z * w$

should be represented as

$$t_1 = z * w$$

$$t_2 = y + t_1$$

$$x = t_2$$

- Where t_1 and t_2 are compiler generated temporary names

Example of 3-address code

a := b * -c + b * -c

```
t1 = - c
t2 = b * t1
t3 = - c
t4 = b * t3
t5 = t2 + t4
a = t5
```

Code for the syntax tree

```
t1 = - c
t2 = b * t1
t5 = t2 + t2
a = t5
```

Code for the DAG

Types of Three-Address Statements

Assignment Statement:

$x = y \text{ op } z$

Assignment Statement:

$x = \text{op } z$ (op is unary operator)

Copy Statement:

$x = z$

Unconditional Jump:

goto L

Conditional Jump:

if x relop y goto L

Stack Operations:

Push/pop

Procedure (function):

param x_1

param x_2

...

param x_n

call p,n

Index Assignments:

$x = y[i]$

$x[i] = y$

Address and Pointer Assignments:

$x = \&y$ $x = *y$ $*x = y$

Implementations of 3-address statements

- In compiler, The Three Address statements are implemented using following representations:
- **Quadruples**
- **Triples**
- **Indirect triples**
- A quadruple is a record with four fields , op , arg1, arg2 and result.

op	arg1	arg2	result
----	------	------	--------

Implementations of 3-address statements using quadruple

Assignment Statement:

$x = y \text{ op } z$

op	arg1	arg2	result
op	y	z	x

Assignment Statement:

$x = \text{op } z$ (op is unary operator)

op	arg1	arg2	result
op	z		x

Copy Statement:

$x = z$

op	arg1	arg2	result
=	z		x

Implementations of 3-address statements using quadruple

a := b * -c + b * -c

- 3-address code

$t_1 = -c$

$t_2 = b * t_1$

$t_3 = -c$

$t_4 = b * t_3$

$t_5 = t_2 + t_4$

$a = t_5$

	<i>op</i>	<i>arg1</i>	<i>arg2</i>	<i>result</i>
(0)	uminus	c		t_1
(1)	*	b	t_1	t_2
(2)	uminus	c		t_3
(3)	*	b	t_3	t_4
(4)	+	t_2	t_4	t_5
(5)	=	t_5		a

Temporary names must be entered into the symbol table as they are created.

Implementations of 3-address statements Triples

- 3-address code

$t_1 = -c$

$t_2 = b * t_1$

$t_3 = -c$

$t_4 = b * t_3$

$t_5 = t_2 + t_4$

$a = t_5$

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

Temporary names are not entered into the symbol table.

Other types of 3-address statements

- e.g. ternary operations like

$x[i] = y$

$x = y[i]$

- require two or more entries. e.g.

$x[i] = y$

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	[] =	x	i
(1)	assign	(0)	y

$x = y[i]$

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(0)	= []	y	i
(1)	assign	x	(0)

Implementations of 3-address statements

- Indirect Triples

	<i>statement</i>
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

triple

	<i>op</i>	<i>arg1</i>	<i>arg2</i>
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	assign	a	(18)

Symbol Table

The Symbol Table

- Used during all phases of compilation
- The symbol table keep track of scope and binding information about names
- For an identifier the possible information includes

name

data type

how it's defined (as a variable, type, function name, etc.)

4. Basic operations performed on symbol table are

- Enter new information
- Look up existing information
- Deleting information

Implementation of symbol table

Two methods of implementing symbol table are

- Linear list

- Hash table

Linear List:

In Linear list, we use an array to store the names and their associated information

The position of the end of the array is marked by the pointer
[available](#)

Implementation of symbol table (contd..)

Hash table:

There are two parts to the data structure

- i) A Hash table consisting of a fixed array of m pointers to table entries.
- ii) Table entries organized into m separate linked lists, called buckets.

To determine whether there is an entry for string s in the symbol table, we apply hash function h to s

such that $h(s)$ returns an integer between 0 to $m-1$

CODE OPTIMIZATION

Machine-independent Code Optimization

- Intermediate code generation process introduces many inefficiencies
 - Extra copies of variables, using variables instead of constants, repeated evaluation of expressions, etc.
- Code optimization removes such inefficiencies and improves code
- Improvement may be time, space, or power consumption
- It changes the structure of programs, sometimes of beyond recognition
 - Inlines functions, unrolls loops, eliminates some programmer-defined variables, etc.
- Code optimization consists of a bunch of heuristics and percentage of improvement depends on programs (may be zero also)

Examples of Machine-Independent Optimizations

- Global common sub-expression elimination
- Copy propagation
- Constant propagation and constant folding
- Loop invariant code motion
- Induction variable elimination and strength reduction
- Partial redundancy elimination
- Loop unrolling
- Function inlining

Data-flow Analysis

- Code optimization needs information about the program
 - which expressions are being recomputed in a function?
 - Which expressions are partially redundant?
 - which definitions reach a point?
 - Which copies and constants can be propagated? Etc.
- All such information is gathered through data-flow analysis

```

void quicksort(int m, int n)
    /* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x; /* swap a[i], a[j] */
    }
    x = a[i]; a[i] = a[n]; a[n] = x; /* swap a[i], a[n] */
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}

```

Three Address code

(1)	$i = m - 1$	(16)	$t7 = 4 * i$
(2)	$j = n$	(17)	$t8 = 4 * j$
(3)	$t1 = 4 * n$	(18)	$t9 = a[t8]$
(4)	$v = a[t1]$	(19)	$a[t7] = t9$
(5)	$i = i + 1$	(20)	$t10 = 4 * j$
(6)	$t2 = 4 * i$	(21)	$a[t10] = x$
(7)	$t3 = a[t2]$	(22)	goto (5)
(8)	if $t3 < v$ goto (5)	(23)	$t11 = 4 * i$
(9)	$j = j - 1$	(24)	$x = a[t11]$
(10)	$t4 = 4 * j$	(25)	$t12 = 4 * i$
(11)	$t5 = a[t4]$	(26)	$t13 = 4 * n$
(12)	if $t5 > v$ goto (9)	(27)	$t14 = a[t13]$
(13)	if $i \geq j$ goto (23)	(28)	$a[t12] = t14$
(14)	$t6 = 4 * i$	(29)	$t15 = 4 * n$
(15)	$x = a[t6]$	(30)	$a[t15] = x$

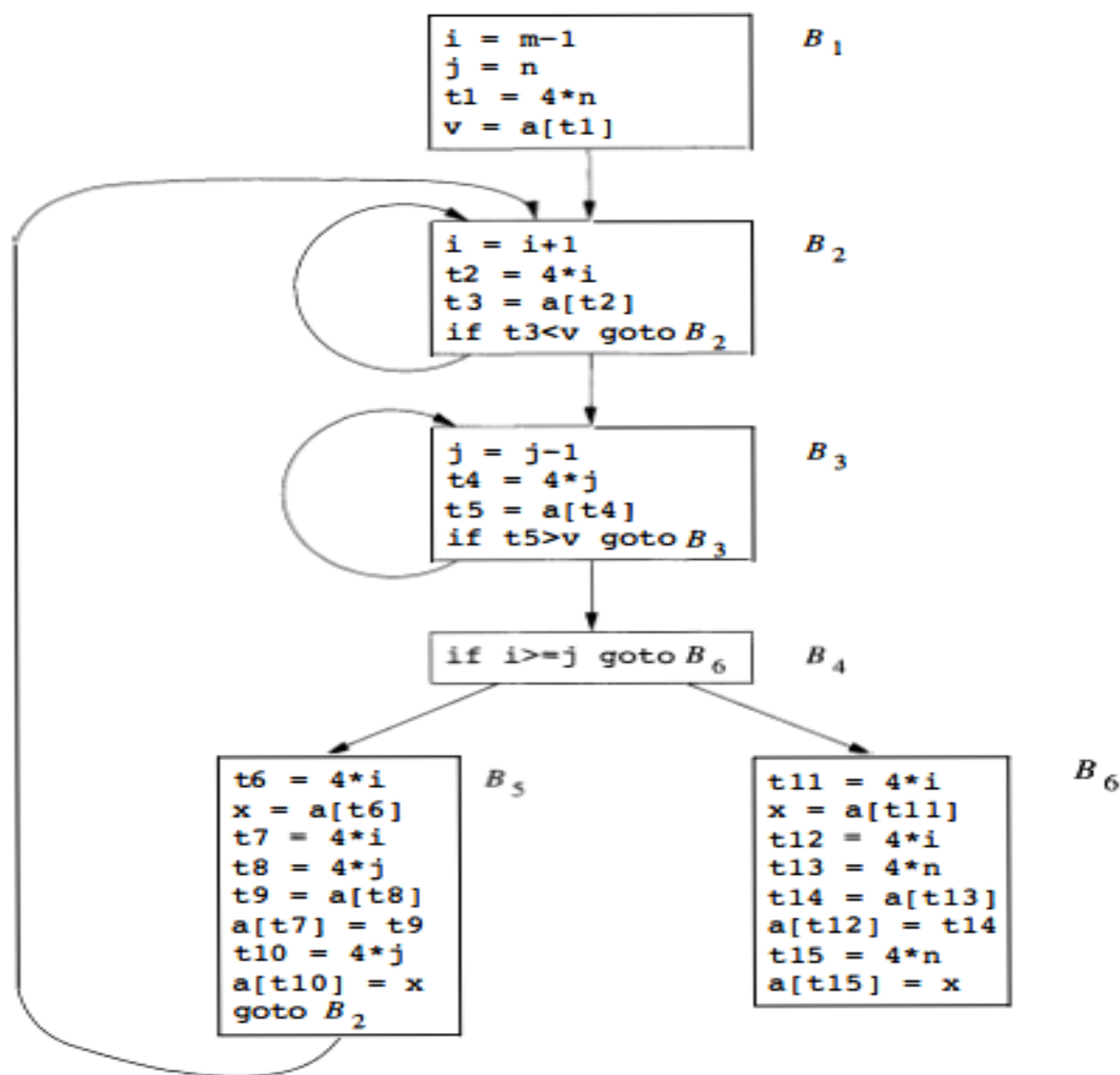
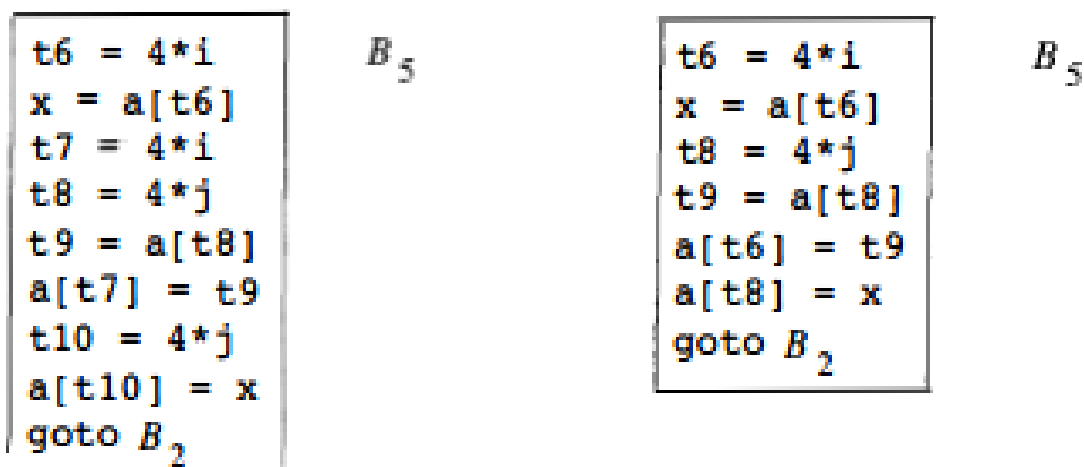


Figure 9.3: Flow graph for the quicksort fragment

Frequently, a program will include several calculations of the same value, such as an offset in an array. As mentioned in Section 9.1.2, some of these duplicate calculations cannot be avoided by the programmer because they lie below the level of detail accessible within the source language. For example, block B_5 shown in Fig. 9.4(a) recalculates $4*i$ and $4*j$, although none of these calculations were requested explicitly by the programmer.



(a) Before.

(b) After.

Figure 9.4: Local common-subexpression elimination

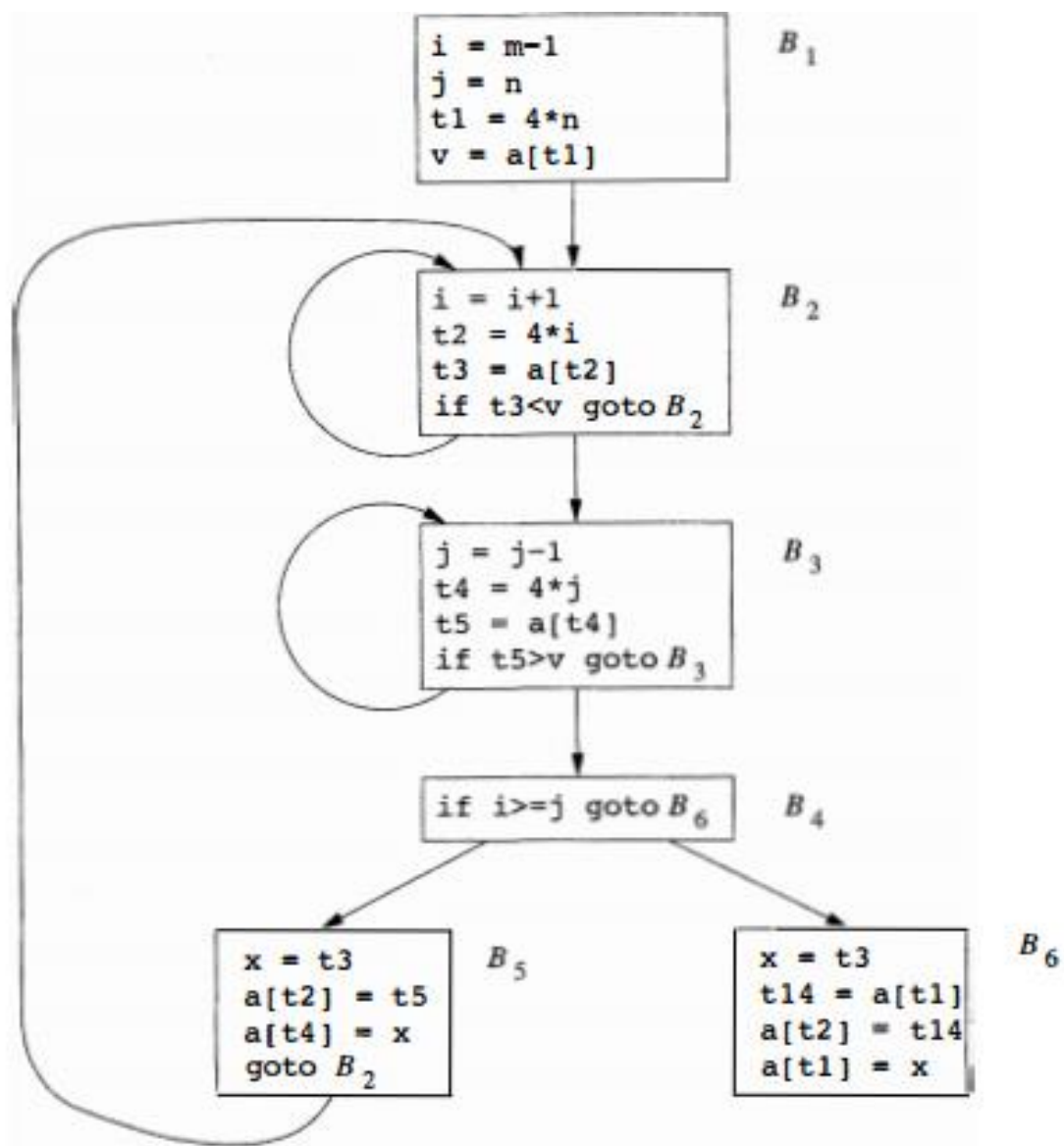


Figure 9.5: B_5 and B_6 after common-subexpression elimination

9.1.5 Copy Propagation

Block B_5 in Fig. 9.5 can be further improved by eliminating x , using two new transformations. One concerns assignments of the form $u = v$ called *copy statements*, or *copies* for short. Had we gone into more detail in Example 9.2, copies would have arisen much sooner, because the normal algorithm for eliminating common subexpressions introduces them, as do several other algorithms.

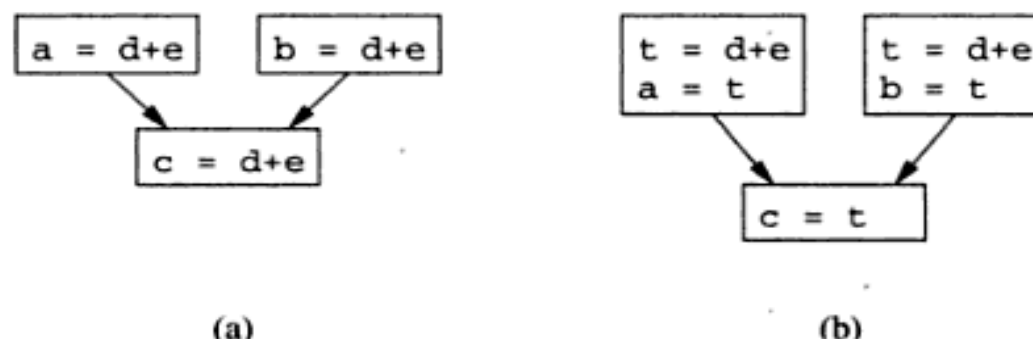


Figure 9.6: Copies introduced during common subexpression elimination

Example 9.3: In order to eliminate the common subexpression from the statement $c = d + e$ in Fig. 9.6(a), we must use a new variable t to hold the value of $d + e$. The value of variable t , instead of that of the expression $d + e$, is assigned to c in Fig. 9.6(b). Since control may reach $c = d + e$ either after the assignment to a or after the assignment to b , it would be incorrect to replace $c = d + e$ by either $c = a$ or by $c = b$. $\square$

9.1.6 Dead-Code Elimination

A variable is *live* at a point in a program if its value can be used subsequently; otherwise, it is *dead* at that point. A related idea is *dead* (or *useless*) *code* — statements that compute values that never get used. While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations.

Example 9.4: Suppose `debug` is set to `TRUE` or `FALSE` at various points in the program, and used in statements like

```
if (debug) print ...
```

It may be possible for the compiler to deduce that each time the program reaches this statement, the value of `debug` is `FALSE`. Usually, it is because there is one particular statement

```
debug = FALSE
```

that must be the last assignment to `debug` prior to any tests of the value of `debug`, no matter what sequence of branches the program actually takes. If copy propagation replaces `debug` by `FALSE`, then the `print` statement is dead because it cannot be reached. We can eliminate both the test and the `print` operation from the object code. More generally, deducing at compile time that the value of an expression is a constant and using the constant instead is known as *constant folding*. $\square$

9.1.7 Code Motion

Loops are a very important place for optimizations, especially the inner loops where programs tend to spend the bulk of their time. The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.

An important modification that decreases the amount of code in a loop is *code motion*. This transformation takes an expression that yields the same result independent of the number of times a loop is executed (a *loop-invariant computation*) and evaluates the expression before the loop. Note that the notion “before the loop” assumes the existence of an entry for the loop, that is, one basic block to which all jumps from outside the loop go (see Section 8.4.5).

Example 9.5: Evaluation of $limit - 2$ is a loop-invariant computation in the following while-statement:

```
while (i <= limit-2) /* statement does not change limit */
```

Code motion will result in the equivalent code

```
t = limit-2
while (i <= t) /* statement does not change limit or t */
```

Now, the computation of $limit - 2$ is performed once, before we enter the loop. Previously, there would be $n + 1$ calculations of $limit - 2$ if we iterated the body of the loop n times. $\square$

9.1.8 Induction Variables and Reduction in Strength

Another important optimization is to find induction variables in loops and optimize their computation. A variable x is said to be an “induction variable” if there is a positive or negative constant c such that each time x is assigned, its value increases by c . For instance, i and $t2$ are induction variables in the loop containing B_2 of Fig. 9.5. Induction variables can be computed with a single increment (addition or subtraction) per loop iteration. The transformation of replacing an expensive operation, such as multiplication, by a cheaper one, such as addition, is known as *strength reduction*. But induction variables not only allow us sometimes to perform a strength reduction; often it is possible to eliminate all but one of a group of induction variables whose values remain in lock step as we go around the loop.

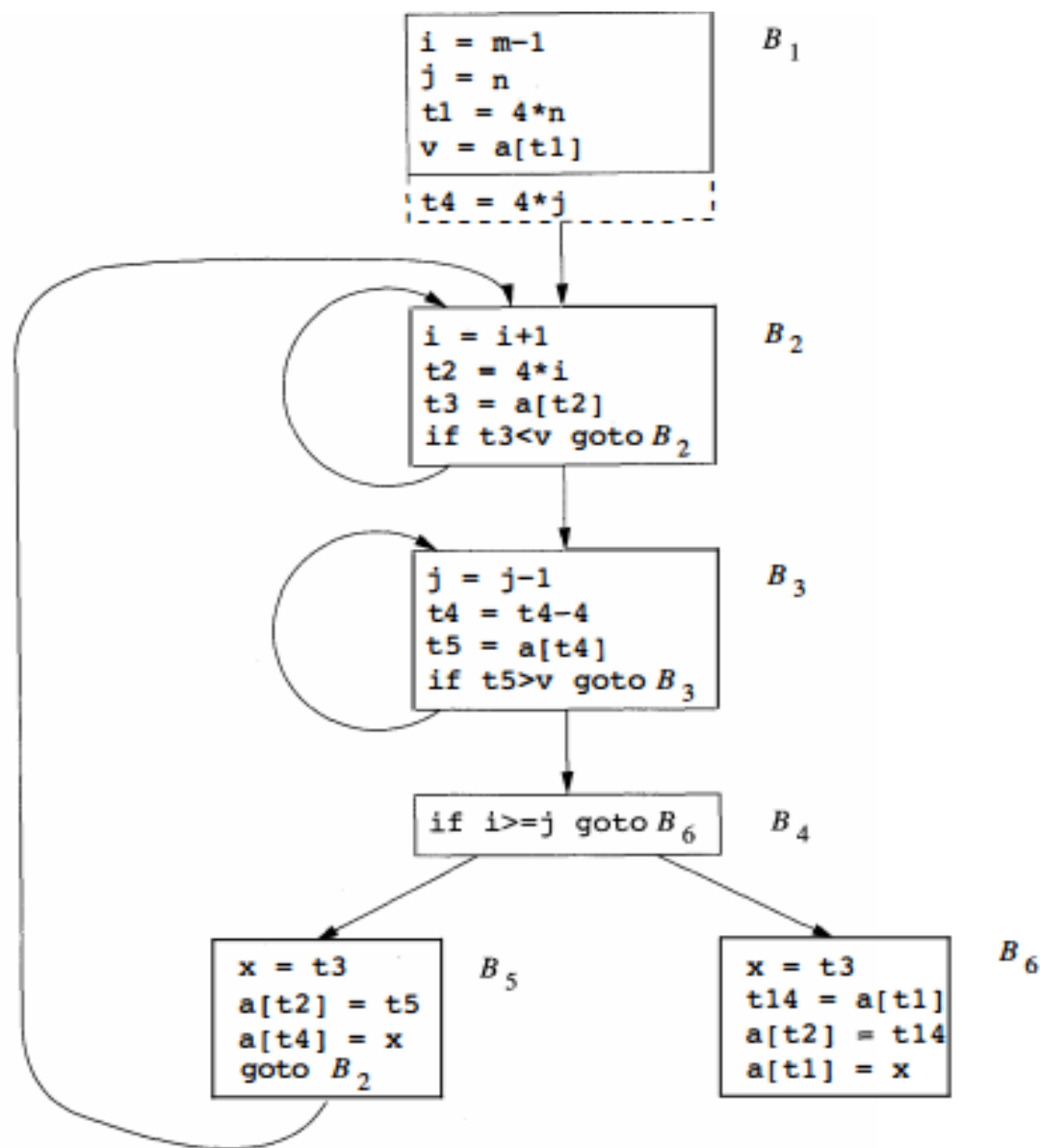


Figure 9.8: Strength reduction applied to $4*j$ in block B_3

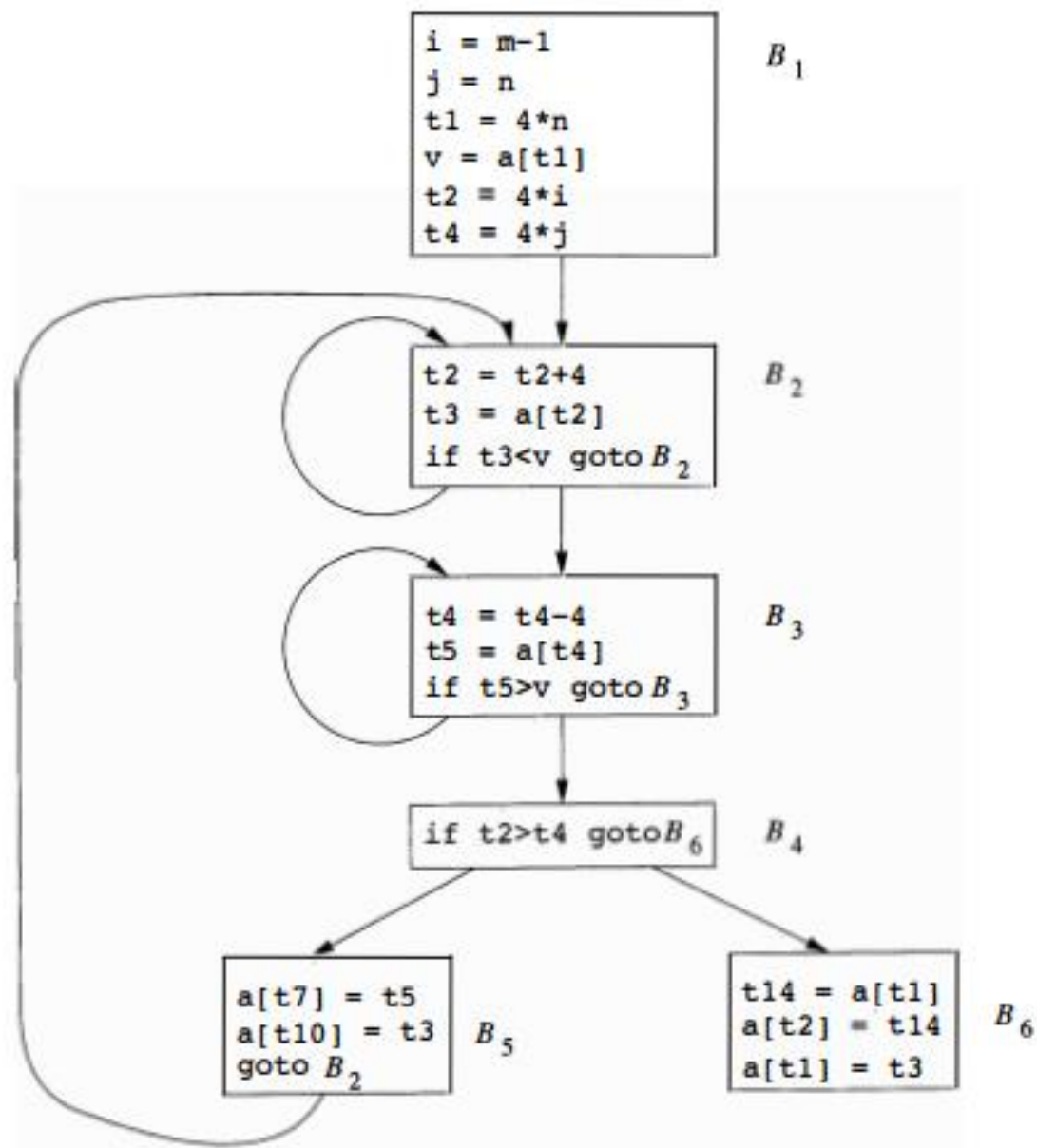


Figure 9.9: Flow graph after induction-variable elimination

Basic Blocks and Flow Graphs

The representation is constructed as follows:

1. Partition the intermediate code into *basic blocks*, which are maximal sequences of consecutive three-address instructions with the properties that
 - (a) The flow of control can only enter the basic block through the first instruction in the block. That is, there are no jumps into the middle of the block.
 - (b) Control will leave the block without halting or branching, except possibly at the last instruction in the block.
2. The basic blocks become the nodes of a *flow graph*, whose edges indicate which blocks can follow which other blocks.

Flow Graphs

Once an intermediate-code program is partitioned into basic blocks, we represent the flow of control between them by a flow graph. The nodes of the flow graph are the basic blocks. There is an edge from block B to block C if and only if it is possible for the first instruction in block C to immediately follow the last instruction in block B . There are two ways that such an edge could be justified:

- There is a conditional or unconditional jump from the end of B to the beginning of C .
- C immediately follows B in the original order of the three-address instructions, and B does not end in an unconditional jump.

We say that B is a *predecessor* of C , and C is a *successor* of B .

Optimization of Basic Blocks

We can often obtain a substantial improvement in the running time of code merely by performing *local* optimization within each basic block by itself.

The DAG Representation of Basic Blocks

Many important techniques for local optimization begin by transforming a basic block into a DAG (directed acyclic graph).

The DAG representation of a basic block lets us perform several code-improving transformations on the code represented by the block.

- a) We can eliminate *local common subexpressions*, that is, instructions that compute a value that has already been computed.
- b) We can eliminate *dead code*, that is, instructions that compute a value that is never used.
- c) We can reorder statements that do not depend on one another; such reordering may reduce the time a temporary value needs to be preserved in a register.
- d) We can apply algebraic laws to reorder operands of three-address instructions, and sometimes thereby simplify the computation.

A DAG for the block

$a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$

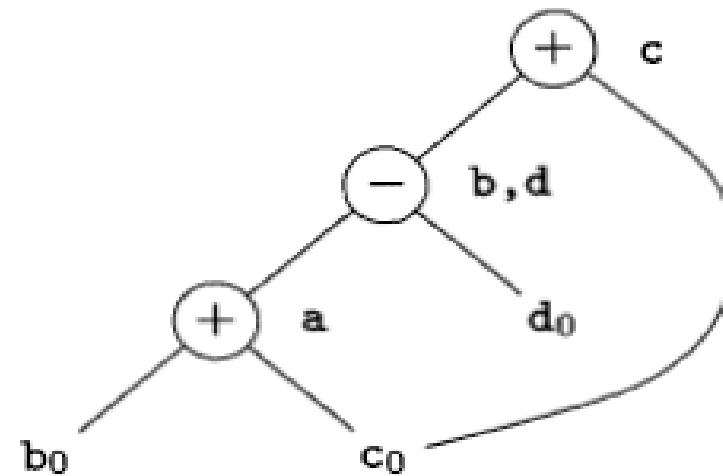


Figure 8.12: DAG for basic block in Example 8.10

It might appear that, since there are only three nonleaf nodes in the DAG of Fig. 8.12, the basic block in Example 8.10 can be replaced by a block with only three statements. In fact, if **b** is not live on exit from the block, then we do not need to compute that variable, and can use **d** to receive the value represented by the node labeled $-$. in Fig. 8.12. The block then becomes

a = **b** + **c**

d = **a** - **d**

c = **d** + **c**

The Use of Algebraic Identities

Algebraic identities represent another important class of optimizations on basic blocks. For example, we may apply arithmetic identities, such as

$$\begin{array}{ll} x + 0 = 0 + x = x & x - 0 = x \\ x \times 1 = 1 \times x = x & x/1 = x \end{array}$$

to eliminate computations from a basic block.

Another class of algebraic optimizations includes local *reduction in strength*, that is, replacing a more expensive operator by a cheaper one as in:

EXPENSIVE		CHEAPER
x^2	=	$x \times x$
$2 \times x$	=	$x + x$
$x/2$	=	$x \times 0.5$

A third class of related optimizations is *constant folding*. Here we evaluate constant expressions at compile time and replace the constant expressions by their values.² Thus the expression $2 * 3.14$ would be replaced by 6.28.

Peephole Optimization

A simple but effective technique for locally improving the target code is *peephole optimization*, which is done by examining a sliding window of target instructions (called the *peephole*) and replacing instruction sequences within the peephole by a shorter or faster sequence, whenever possible. Peephole optimization can also be applied directly after intermediate code generation to improve the intermediate representation.

The peephole is a small, sliding window on a program. The code in the peephole need not be contiguous, although some implementations do require this. It is characteristic of peephole optimization that each improvement may spawn opportunities for additional improvements. In general, repeated passes over the target code are necessary to get the maximum benefit. In this section, we shall give the following examples of program transformations that are characteristic of peephole optimizations:

- Redundant-instruction elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms

Eliminating Redundant Loads and Stores

If we see the instruction sequence

```
LD  a, R0  
ST  R0, a
```

in a target program, we can delete the store instruction because whenever it is executed, the first instruction will ensure that the value of **a** has already been loaded into register **R0**. Note that if the store instruction had a label, we could not be sure that the first instruction is always executed before the second, so we could not remove the store instruction. Put another way, the two instructions have to be in the same basic block for this transformation to be safe.

Eliminating Unreachable Code

Another opportunity for peephole optimization is the removal of unreachable instructions. An unlabeled instruction immediately following an unconditional jump may be removed. This operation can be repeated to eliminate a sequence of instructions. For example, for debugging purposes, a large program may have within it certain code fragments that are executed only if a variable **debug** is equal to 1. In the intermediate representation, this code may look like

```
    if debug == 1 goto L1
    goto L2
L1: print debugging information
L2:
```

One obvious peephole optimization is to eliminate jumps over jumps. Thus, no matter what the value of **debug**, the code sequence above can be replaced by


```
    if debug != 1 goto L2
    print debugging information
L2:
```

If **debug** is set to 0 at the beginning of the program, constant propagation would transform this sequence into

```
    if 0 != 1 goto L2
    print debugging information
L2:
```

Now the argument of the first statement always evaluates to *true*, so the statement can be replaced by **goto L2**. Then all statements that print debugging information are unreachable and can be eliminated one at a time.

Flow-of-Control Optimizations

Simple intermediate code-generation algorithms frequently produce jumps to jumps, jumps to conditional jumps, or conditional jumps to jumps. These unnecessary jumps can be eliminated in either the intermediate code or the target code by the following types of peephole optimizations. We can replace the sequence

```
    goto L1
    ...
L1: goto L2
```

by the sequence

```
    goto L2
    ...
L1: goto L2
```

If there are now no jumps to L1, then it may be possible to eliminate the statement L1: goto L2 provided it is preceded by an unconditional jump.

Algebraic Simplification and Reduction in Strength

we discussed algebraic identities that could be used to simplify DAG's. These algebraic identities can also be used by a peephole optimizer to eliminate three-address statements such as

$$x = x + 0$$

or

$$x = x * 1$$

in the peephole.

Similarly, reduction-in-strength transformations can be applied in the peephole to replace expensive operations by equivalent cheaper ones on the target machine. Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators. For example, x^2 is invariably cheaper to implement as $x * x$ than as a call to an exponentiation routine. Fixed-point multiplication or division by a power of two is cheaper to implement as a shift. Floating-point division by a constant can be approximated as multiplication by a constant, which may be cheaper.

Use of Machine Idioms

The target machine may have hardware instructions to implement certain specific operations efficiently. Detecting situations that permit the use of these instructions can reduce execution time significantly. For example, some machines have auto-increment and auto-decrement addressing modes. These add or subtract one from an operand before or after using its value. The use of the modes greatly improves the quality of code when pushing or popping a stack, as in parameter passing. These modes can also be used in code for statements like $x = x + 1$.